MLP Project Milestone 4: Model Building and Hyperparameter Exploration

To understand about all the milestones in detail, Please *click here*

Roll number	

Submission guidelines

1. **Make your own copy with File > Make a copy, and write only in your copy.**
2. **Once you have finished writing your report, save or export the .docx document as a PDF.**
3. **Rename the PDF file using the following exact convention: [roll-number]_26t3_M[Milestone-Number].pdf**
4. **Example: If your roll number is 21f1001234 and you are submitting the PDF for Milestone 4, your file must be named exactly: 21f1001234_26t3_M4.pdf**

Instructions

- Use the Project data to answer the questions.
- Write each answer in the "Your answer" box under the question, and paste the Python code (wherever required) you ran into the "Your code" box as text, not as a screenshot.
- Support every claim with evidence from your own analysis of the competition data: a number, a small table or a plot (Insert > Image). Summarise long outputs instead of pasting them in full.
- The points under each question describe what an answer should cover. If a point does not apply to your dataset (for example, there are no date columns), you can skip it.
- Write in your own words. Clear reasoning backed by evidence matters more than length of your answer.
- There is no single correct answer, these questions are subjective and there can be multiple valid explanations of questions. Use your interpretation and understanding to answer the questions.
- You need not include all the columns(features) while answering questions, you can select a few meaningful columns and do analysis on them. Try to cover at least 50% of the columns(features) for the questions in this milestone.
- You MAY use a different kaggle notebook instead of the one shared with the host to do the task given in milestones.
- The answer length can vary depending on questions. Keep your answer concise and give a summary of the analysis and cover details using visualizations/bullet points wherever required. The maximum length of answer can be considered around 100 to 400 words.
- **Plots, graphs and figures:** add them in the "Figures" section at the end of this document, under the heading of the question they belong to, not inside the answer boxes. Number

each figure as Figure Q<question number>.<figure number> (for example, Figure Q3.1) and refer to it by this number in your answer (for example, "see Figure Q3.1").

Q1. Model Building Plan

Identify suitable models for the given prediction problem and explain the planned model-building approach.

One suggested approach would be as follows:

- Identify suitable linear, non-tree, tree-based, and ensemble models for the prediction problem, where applicable.
- Justify the selection of models based on the nature of the target variable, dataset characteristics, and relevant modelling assumptions.

Problem. Predict price_tier (1 = cheapest … 4 = most expensive) for 114,538 test spaces from 59 features (operator, location, capacity, booking terms, availability, reviews, facilities). The target is a **4-class ordinal label** with fairly balanced classes (23.9% / 29.1% / 21.7% / 25.2% over 279,746 rows), and the metric is **macro-F1**, so every tier matters equally.

Data characteristics that drive model choice

• Mixed types: ~50 numeric, 7 categorical (3 high-cardinality: zone 1,380 levels, space_class 131, borough 55), a JSON list of 144 facility codes, and date columns.

• Structured missingness: 19.4% of rows have no reviews (all score_* missing together), and borough is 76.7% missing.

• Strong non-linear and interaction effects: median max_seats is 2/2/4/5 by tier, and 85% of shared_zone spaces are tier 1 but only 17% of whole_unit ones.

• Large n (280k): KNN and SVM become costly, while tree ensembles scale well.

Selected models

• **Linear:** multinomial Logistic Regression, an interpretable baseline that shows how far linear boundaries go.

• **Non-tree, non-linear:** K-Nearest Neighbours, which assumes that similar spaces (location, size) cost alike.

• **Tree:** Decision Tree, which captures thresholds and interactions without scaling.

• **Bagging:** Random Forest, which cuts the variance of single trees.

• **Boosting:** LightGBM and XGBoost, the strongest family on large tabular data with native missing-value handling.

• **Ensembles:** soft Voting and Stacking, which combine families whose errors differ.

Plan. Shared stateless feature engineering, then one sklearn Pipeline per model (target encoding, scaling and imputation fitted inside CV folds to avoid leakage). Baselines are scored on a stratified 80/20 hold-out, models are tuned with stratified 3-fold CV on a 60k sample, and before/after scores are compared on the same hold-out. Initial hold-out macro-F1: LightGBM 0.744, Random Forest 0.721, Logistic Regression 0.659, Decision Tree 0.615, KNN 0.570 (see Figure Q1.1–Q1.2 for the EDA evidence).

```
X_full, X_test = build_features(train_raw), build_features(test_raw)   # stateless feature
engineering
X_tr, X_val, y_tr, y_val = train_test_split(X_full, y_full, test_size=0.2, stratify=y_full,
random_state=42)

baseline_models = {
    'Logistic Regression': make_pipeline(make_linear_preprocessor(),
LogisticRegression(max_iter=1000)),
    'KNN (50k sample)':    make_pipeline(make_linear_preprocessor(), KNeighborsClassifier()),
    'Decision Tree':       make_pipeline(make_tree_preprocessor(),
DecisionTreeClassifier(random_state=42)),
    'Random Forest':       make_pipeline(make_tree_preprocessor(),
RandomForestClassifier(n_estimators=100)),
    'LightGBM':            make_pipeline(make_tree_preprocessor(impute=False), lgb.LGBMClassifier()),
    'XGBoost':             make_pipeline(make_tree_preprocessor(impute=False),
                                         EncodeLabels(xgb.XGBClassifier(tree_method='hist'))),
}
for name, model in baseline_models.items():
    X_fit, y_fit = (X_tr.loc[knn_idx], y_tr.loc[knn_idx]) if name.startswith('KNN') else (X_tr, y_tr)
    model.fit(X_fit, y_fit)
    print(name, f1_score(y_val, model.predict(X_val), average='macro'))
```

Q2. Linear Models

Explore linear models and describe how their assumptions and characteristics influence their behavior on the given dataset. Implement and explore at least one suitable model, identify and describe the role of hyperparameters of the model.

One suggested approach would be as follows:

- Explain the fundamental assumptions of the selected linear model and how it represents the relationship between input features and the target variable.
- Discuss the impact of preprocessing steps on the model. Describe the strength and limitations of the model based on the dataset.
- Explain the role of relevant hyperparameters of the selected model and describe how they influence the model.

Model. Multinomial Logistic Regression models the log-odds of each tier as a linear function of the features, $z_k = w_k \cdot x + b_k$, turns these into probabilities with softmax, and minimises cross-entropy plus an L2 penalty (lbfgs solver).

Assumptions: each feature has an additive, monotonic effect on the log-odds; classes are separable by hyperplanes; observations are independent; there is no severe multicollinearity.

Behaviour on this data. Hold-out macro-F1 = **0.659**, well below LightGBM (0.744). The model can say "more seats means a more expensive tier", but not "seats matter differently in shared vs whole units" or price jumps between metros. Those interactions are exactly where it loses. Tier 1 vs 2 (similar capacity, median max_seats 2 for both) is the hardest boundary for it.

Preprocessing impact (essential for this model):

- **Standard scaling** puts all features on one scale, so the L2 penalty treats them equally and lbfgs converges.
- **log1p** on 26 right-skewed counts (review_count, operator_portfolio_size, max_term_days, …) stops extreme values dominating the linear score.
- **One-hot** encoding for low-cardinality categoricals (access_type, metro, …), **target encoding** for zone/space_class/borough (one-hot would add ~1,500 sparse columns), and frequency encoding for operator_ref.
- **Median imputation with missing-indicator columns**, so "never reviewed" becomes a usable signal.

Strengths: fast (~27 s on 224k rows), interpretable coefficients, calibrated probabilities that are useful in a stacking ensemble.

Limitations: cannot model interactions or thresholds without manual feature crosses, and is sensitive to scaling and outliers.

Hyperparameters

- **C** (inverse regularisation strength): small C shrinks weights (more bias), large C fits more closely (more variance). Grid 0.001–10 on a log scale.
- **class_weight**: 'balanced' re-weights the loss towards smaller tiers, which can lift macro-F1.
- **penalty / solver**: L2 with lbfgs; L1 (saga) would zero out redundant features such as the open_slots_* group.
- **max_iter**: must be high enough to converge.

```
def make_linear_preprocessor():
    log_scale = Pipeline([('impute', SimpleImputer(strategy='median')),
                          ('log', FunctionTransformer(np.log1p)), ('scale', StandardScaler())])
    plain = Pipeline([('impute', SimpleImputer(strategy='median', add_indicator=True)), ('scale',
StandardScaler())])
    return ColumnTransformer([
        ('target_enc', make_pipeline(TargetEncoder(random_state=42), StandardScaler()), HIGH_CARD),
        ('freq_enc', make_pipeline(FrequencyEncoder(), FunctionTransformer(np.log1p),
StandardScaler()), FREQ_COLS),
        ('onehot', OneHotEncoder(handle_unknown='ignore'), LOW_CARD),
        ('log_num', log_scale, SKEWED), ('num', plain, PLAIN)])

lr_pipe = make_pipeline(make_linear_preprocessor(), LogisticRegression(max_iter=2000))
lr_grid = {'logisticregression__C': [0.001, 0.01, 0.1, 1, 10],
           'logisticregression__class_weight': [None, 'balanced']}
lr_search = GridSearchCV(lr_pipe, lr_grid, scoring='f1_macro', cv=StratifiedKFold(3, shuffle=True,
random_state=42))
lr_search.fit(X_tune, y_tune)
# preprocessing ablation: no scaling / scaling without log / log + scaling
for label, pre in [('no scaling', no_scale), ('scaling, no log', no_log), ('log + scaling',
make_linear_preprocessor())]:
    print(label, cross_val_score(lr_variant(pre), X_tune, y_tune, scoring='f1_macro', cv=CV3).mean())
best_lr = clone(lr_pipe).set_params(**lr_search.best_params_).fit(X_tr, y_tr)
```

Q3. Other Non-Tree Models

Build a suitable non-tree and non linear model for the given prediction problem. Choose at least one model and describe the underlying assumptions, mechanisms and behavior on the given dataset. Discuss the impact of preprocessing, identify and describe the role of hyperparameters of the model.

One suggested approach would be as follows:

- Explain the fundamental assumptions of the selected model and how it represents the relationship between input features and the target variable.
- Discuss the impact of preprocessing steps on the model. Describe the strength and limitations of the model based on the dataset.
- Explain the role of relevant hyperparameters of the selected model and describe how they influence the model.

Model. K-Nearest Neighbours (non-parametric, non-linear). It stores the training set and labels a new space with the majority tier of its **k** closest training points (Euclidean distance), optionally weighting neighbours by 1/distance.

Assumption: spaces that are close in feature space (similar location, size, terms) have similar price tiers. It makes no assumption about the shape of the decision boundary, so it can follow irregular boundaries that logistic regression cannot.

Behaviour on this data. With default k = 5, trained on a 50k-row sample, hold-out macro-F1 = **0.570**, the weakest model. Reasons:

- **Curse of dimensionality:** after encoding there are ~230 dimensions, including ~135 binary facility flags. Distances become less informative, and many irrelevant features dilute the few strong ones (max_seats, location).
- All features get equal weight in the distance, unlike trees, which select features.
- k = 5 is noisy for overlapping tiers 1 vs 2; a larger k should help.

Preprocessing impact (the biggest of any model)

- **Scaling is mandatory**: without it open_slots_d (0–365) or geo_lon would dominate the distance over binary flags.
- log1p on skewed counts keeps outliers from creating huge distances.
- Imputation is required (KNN cannot handle NaN).
- Target encoding keeps the dimensionality manageable.

Strengths: no training phase, captures local non-linear structure, few assumptions.

Limitations: prediction cost is O(n_train × n_test × d), so I trained on a 50k sample to keep it tractable. It is sensitive to irrelevant features and scaling, and gives no feature importances.

Hyperparameters

- **n_neighbors (k)**: small k means low bias and high variance (noisy boundaries); large k smooths but blurs tiers. Grid 5–81.
- **weights**: 'distance' lets closer neighbours count more, which usually helps when classes overlap.
- **metric / p**: Euclidean (p = 2) vs Manhattan (p = 1).
- **Training-set size / algorithm**: brute force is fastest in ~230 dimensions.

```
knn_pipe = make_pipeline(make_linear_preprocessor(), KNeighborsClassifier(algorithm='brute'))
knn_grid = {'kneighborsclassifier__n_neighbors': [5, 11, 21, 41, 81],
            'kneighborsclassifier__weights': ['uniform', 'distance']}
knn_search = GridSearchCV(knn_pipe, knn_grid, scoring='f1_macro', cv=CV3).fit(X_tune.iloc[:30000],
y_tune.iloc[:30000])

for n in [10000, 25000, 50000, 100000]:            # effect of training-set size
    idx = X_tr.sample(n, random_state=42).index
    m = clone(knn_pipe).set_params(**knn_search.best_params_).fit(X_tr.loc[idx], y_tr.loc[idx])
    print(n, f1_score(y_val, m.predict(X_val), average='macro'))
```

Q4. Decision Trees

Explore decision trees and understand how they learn from data. Describe the underlying assumptions, mechanisms and behavior on the given dataset. Discuss the impact of preprocessing, identify and describe the role of hyperparameters of the model.

One suggested approach would be as follows:

- Explain the process of the decision tree model and how it represents the relationship between input features and the target variable.
- Discuss the impact of preprocessing steps on the model. Describe the strength and limitations of the model based on the dataset.
- Explain the role of relevant hyperparameters of the selected model and describe how they influence the model.

Process. A CART decision tree recursively splits the data. At each node it tries every feature and threshold and keeps the split that most reduces impurity (Gini = $1 - \Sigma p_k^2$, or entropy), then repeats on each child until a stopping rule fires. Each leaf predicts its majority tier. The result is a set of if-then rules, e.g. max_seats ≤ 3 → access_type = shared_zone → tier 1. The trained model is the tree itself: piecewise-constant regions in feature space, with interactions captured naturally because later splits depend on earlier ones.

Behaviour on this data

- A fully grown tree (default) reaches hold-out macro-F1 **0.615**. It memorises the 224k training rows, so it overfits.
- Simply limiting it (max_depth = 15, min_samples_leaf = 20) raises this to **0.649**, a +0.034 gain from regularisation alone and close to logistic regression (0.659) with a single tree.
- Early splits use capacity (max_seats, workstations), access type and location (target-encoded zone, metro), matching the EDA.

Preprocessing impact

- **No scaling or log transforms needed**: splits depend only on the ordering of values, so monotonic transforms change nothing.
- Low-cardinality categoricals can be **ordinal-encoded** rather than one-hot (fewer columns, faster).
- High-cardinality zone is target-encoded so one split can separate expensive from cheap zones.
- Missing values are median-imputed (has_reviews keeps the missingness signal).

Strengths: interpretable, handles interactions and non-linearity, fast (~12–16 s), robust to outliers.

Limitations: high variance (small data changes give a different tree), greedy splits, axis-aligned boundaries, and overfitting without constraints.

Hyperparameters

- **max_depth**: caps the number of rule levels, the main bias–variance control.
- **min_samples_leaf / min_samples_split**: require enough rows per leaf, so noisy, tiny leaves disappear.
- **criterion**: gini vs entropy (usually similar).
- **ccp_alpha**: cost-complexity pruning after growth.
- **max_features**: considers a random subset of features per split.

```
def make_tree_preprocessor(impute=True):
    return ColumnTransformer([
        ('target_enc', TargetEncoder(random_state=42), HIGH_CARD),
        ('freq_enc', FrequencyEncoder(), FREQ_COLS),
        ('ordinal', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1), LOW_CARD),
        ('num', SimpleImputer(strategy='median') if impute else 'passthrough', NUM_COLS)])

dt_pipe = make_pipeline(make_tree_preprocessor(), DecisionTreeClassifier(random_state=42))
dt_grid = {'decisiontreeclassifier__max_depth': [6, 10, 14, 18, 24, None],
           'decisiontreeclassifier__min_samples_leaf': [1, 10, 50, 200],
           'decisiontreeclassifier__criterion': ['gini', 'entropy']}
dt_search = GridSearchCV(dt_pipe, dt_grid, scoring='f1_macro', cv=CV3,
return_train_score=True).fit(X_tune, y_tune)
best_dt = clone(dt_pipe).set_params(**dt_search.best_params_).fit(X_tr, y_tr)
plot_tree(best_dt[-1], max_depth=2, feature_names=list(best_dt[0].get_feature_names_out()),
filled=True)
```

Q5. Bagging Ensembles

Explore bagging-based ensemble models and understand how they learn from data. Describe the underlying mechanisms and behavior of these models on the given dataset. Discuss the impact of preprocessing, identify and describe the role of hyperparameters of the selected model.

One suggested approach would be as follows:

- Explain the working mechanism of the selected bagging model and how it combines multiple base models to make predictions.
- Discuss the impact of preprocessing steps on the model. Describe the strength and limitations of the model based on the dataset.
- Explain the role of relevant hyperparameters of the selected model and describe how they influence the model.

Model. Random Forest. It trains many deep decision trees, each on a **bootstrap sample** of the rows (sampling with replacement, ~63% unique rows per tree) and considering only a **random subset of features** (max_features) at each split. For prediction, the trees' class probabilities are averaged and the most probable tier wins. Bootstrapping plus feature subsampling de-correlate the trees, so averaging keeps the low bias of deep trees while sharply reducing variance. The ~37% of rows a tree did not see (**out-of-bag**) give a free validation estimate.

Behaviour on this data. With 200 trees, hold-out macro-F1 = **0.721**, up from **0.615** for one unpruned tree (+0.106). This is the clearest evidence that the single tree's problem was variance, not bias. It passes the 0.70 cut-off but stays below LightGBM (0.744), because bagging averages independent trees and cannot correct their shared mistakes on the hard tier 1 vs 2 boundary. Training took ~40 s on 4 cores.

Preprocessing impact

- As for trees: no scaling needed, ordinal encoding for low-cardinality categoricals, and target encoding for zone/space_class/borough.
- sklearn forests need imputation (median).
- Many correlated features (open_slots_a…e, min_term_*) are tolerated because feature subsampling spreads importance across them, but this also splits their importance scores.

Strengths: robust, hard to overfit as trees are added, parallel, OOB estimate, feature importances.

Limitations: memory-heavy deep trees, slower prediction, less interpretable, and impurity importances are biased towards high-cardinality/continuous features.

Hyperparameters

- **n_estimators**: more trees give lower variance with diminishing returns and never overfit, only cost time.
- **max_features**: smaller values give more diverse trees (lower correlation) but each tree is weaker. 'sqrt' ≈ 14 of ~190 features.
- **max_depth / min_samples_leaf**: tree complexity; deep trees are fine because averaging removes the variance.
- **class_weight = 'balanced_subsample'**: re-weights tiers within each bootstrap.
- **bootstrap**: on or off.

```
rf_pipe = make_pipeline(make_tree_preprocessor(), RandomForestClassifier(random_state=42))
rf_space = {'randomforestclassifier__n_estimators': [150, 300],
            'randomforestclassifier__max_features': ['sqrt', 0.2, 0.35],
            'randomforestclassifier__max_depth': [None, 20, 30],
            'randomforestclassifier__min_samples_leaf': [1, 2, 5],
            'randomforestclassifier__class_weight': [None, 'balanced_subsample']}
rf_search = RandomizedSearchCV(rf_pipe, rf_space, n_iter=12, scoring='f1_macro', cv=CV3,
random_state=42)
rf_search.fit(X_tune, y_tune)

for n in [10, 25, 50, 100, 200, 300]:              # out-of-bag score vs number of trees
    rf_ = RandomForestClassifier(**dict(best_rf_params, n_estimators=n), oob_score=True).fit(Xt,
y_tune)
    oob_pred = rf_.classes_[rf_.oob_decision_function_.argmax(1)]
    print(n, f1_score(y_tune, oob_pred, average='macro'))
```

Q6. Boosting Ensembles

Explore boosting-based ensemble models and understand how they learn from data. Describe the underlying mechanisms and behavior of these models on the given dataset. Discuss the impact of preprocessing, identify and describe the role of hyperparameters of the selected model.

One suggested approach would be as follows:

- Explain the working mechanism of the selected boosting model and how it is built.
- Discuss the impact of preprocessing steps on the model. Describe the strength and limitations of the model based on the dataset.
- Explain the role of relevant hyperparameters of the selected model and describe how they influence the model.

Model. LightGBM (gradient-boosted decision trees); XGBoost is used as a second implementation. Boosting builds trees **sequentially**. It starts from the class priors; at each round, for each of the 4 classes, a new small tree is fitted to the **negative gradient of the multi-class log-loss** (roughly what the current ensemble still gets wrong), and its output is added with shrinkage: $F_m = F_{m-1} + \eta \cdot \text{tree}_m$. Unlike bagging (independent trees, averaged), every tree corrects its predecessors, so boosting reduces **bias** as well as variance.

LightGBM specifics: histogram binning of features (fast on 280k rows), **leaf-wise** growth (splits the leaf with the largest loss reduction), and native NaN handling (learns a default direction).

Behaviour on this data. Best single model: hold-out macro-F1 = **0.744** (num_leaves = 127, learning_rate = 0.05, subsample 0.8, colsample 0.5, early stopping at 1,156 trees, ~4 min). With learning_rate = 0.1 and a fixed 600 trees it reaches 0.740. It beats Random Forest (0.721) because sequential correction focuses on the hard tier 1 vs 2 and 3 vs 4 boundaries.

Preprocessing impact

- No scaling. Missing values are passed as **NaN** (no imputation needed), so missingness is learned directly.
- Target encoding of high-cardinality categoricals gave the same score as LightGBM's native categorical handling (**0.7404 vs 0.7403**), so I kept the sklearn pipeline for consistency and leakage safety.
- Top features by gain: zone, metro, zones, max_seats, space_class, coordinates and min_term_*.

Strengths: highest accuracy, fast, handles NaN and mixed features, built-in regularisation.

Limitations: many interacting hyperparameters, can overfit without early stopping, sequential (less parallel than RF), less interpretable.

Hyperparameters

- **learning_rate × n_estimators**: smaller steps need more trees but generalise better; early stopping picks the count.
- **num_leaves / max_depth**: tree complexity and interaction order.
- **min_child_samples**: minimum rows per leaf.
- **subsample / colsample_bytree**: randomness that reduces variance.
- **reg_lambda / reg_alpha**: L2/L1 penalties on leaf weights.

```
lgb_pipe = make_pipeline(make_tree_preprocessor(impute=False), lgb.LGBMClassifier(subsample_freq=1,
verbose=-1))
lgb_space = {'lgbmclassifier__n_estimators': [200, 400, 700], 'lgbmclassifier__learning_rate': [0.03,
0.06, 0.1],
             'lgbmclassifier__num_leaves': [31, 63, 127, 255], 'lgbmclassifier__min_child_samples':
[10, 30, 80],
             'lgbmclassifier__subsample': [0.7, 0.85, 1.0], 'lgbmclassifier__colsample_bytree': [0.3,
0.5, 0.8],
             'lgbmclassifier__reg_lambda': [0, 1, 5]}
lgb_search = RandomizedSearchCV(lgb_pipe, lgb_space, n_iter=16, scoring='f1_macro', cv=CV3,
random_state=42)
lgb_search.fit(X_tune, y_tune)

# early stopping on a 10% split of the TRAINING part chooses the number of trees per learning rate
X_in, X_es, y_in, y_es = train_test_split(X_tr, y_tr, test_size=0.1, stratify=y_tr, random_state=42)
for lr in [0.1, 0.05]:
    m = lgb.LGBMClassifier(**dict(best_lgb_params, learning_rate=lr, n_estimators=3000))
    m.fit(Xin_t, y_in, eval_set=[(Xes_t, y_es)], callbacks=[lgb.early_stopping(100)])
```

Q7. Voting and Stacking

Explore voting and stacking ensemble models and understand how the algorithm works. Describe the underlying mechanisms and behavior of these models on the given dataset. Discuss the impact of preprocessing, identify and describe the role of hyperparameters of the selected model.

One suggested approach would be as follows:

- Explain the working mechanisms of voting and stacking models and how they combine predictions from multiple base models to make final predictions.
- Discuss the impact of preprocessing steps on the model. Describe the strength and limitations of the model based on the dataset.
- Explain the role of relevant hyperparameters of the selected model and describe how they influence the model.

Voting. Several already-good models are trained independently on the same data and their predictions are combined:

- **Hard voting:** majority of predicted labels.
- **Soft voting:** a (weighted) average of predicted class probabilities, then argmax. It usually beats hard voting because it uses each model's confidence.

Stacking. The base models' **out-of-fold** predicted probabilities (from internal k-fold CV, so no row is predicted by a model that saw it) become input features for a **meta-learner** (here logistic regression). The meta-learner learns how much to trust each base model, per tier. At predict time, base models refitted on all data feed it.

Design for this data. Base models come from different families so their errors are less correlated: Logistic Regression (linear, 0.659), Random Forest (bagging, 0.721), LightGBM (boosting, 0.744) and XGBoost. Each keeps its **own preprocessing pipeline** inside the ensemble (scaling and one-hot for LR; ordinal and NaN-aware for trees), which sklearn's VotingClassifier/StackingClassifier support because each estimator is a full Pipeline.

Expected behaviour. Gains come only when base models are both strong and diverse. Here the boosting models are clearly ahead, so an equal-weight vote can be pulled down by the weaker LR. Weighting boosting higher, or letting stacking learn the weights, should do better. The hold-out results of the ensembles are **[fill after running]**.

Strengths: often the best leaderboard score, reduces variance of individual models, and stacking learns per-tier trust.

Limitations: cost multiplies (stacking with cv = 3 fits every base model 4 times), the result is less interpretable, and there is a risk of overfitting the meta-learner.

Hyperparameters

- Voting: **voting** ('soft'/'hard'), **weights** per model, and the choice of estimators.
- Stacking: **final_estimator** (and its C), **cv** folds for out-of-fold predictions, **stack_method** (predict_proba), and **passthrough** (also feed raw features to the meta-learner).

```
vote = VotingClassifier([('lr', best_lr), ('rf', best_rf), ('lgb', lgb_es), ('xgb', best_xgb)],
voting='soft')
vote.fit(X_tr, y_tr)
for params in [dict(voting='soft', weights=None), dict(voting='soft', weights=[1, 1, 2, 2]),
               dict(voting='soft', weights=[0, 1, 1, 1]), dict(voting='hard', weights=None)]:
    vote.set_params(**params)                     # used only at predict time: no refit needed
    print(params, f1_score(y_val, vote.predict(X_val), average='macro'))

stack = StackingClassifier(estimators=[('lr', best_lr), ('rf', best_rf), ('lgb', lgb_es)],
                           final_estimator=LogisticRegression(max_iter=2000),
                           cv=StratifiedKFold(3, shuffle=True, random_state=42),
stack_method='predict_proba')
stack.fit(X_tr, y_tr)
print('stacking', f1_score(y_val, stack.predict(X_val), average='macro'))
```

Q8. Hyperparameter Tuning(HPT) Strategy

Explain the process of selecting suitable hyperparameter configurations for machine learning models. Explore how cross-validation is used to obtain reliable estimates of model performance and how different search strategies can be used to identify suitable configurations.

One suggested approach would be as follows:

- Explain the factors that should be considered when selecting hyperparameters and defining their search ranges. Discuss how the choice of hyperparameter values can influence the trade-off between model complexity and generalisation.
- Explore different cross-validation strategies and describe the importance and implementation steps of them.
- Explore HPT approaches. Explain how they explore the search space and compare their advantages, limitations, and computational requirements.

Selecting hyperparameters and ranges

- Start from the parameters that control **model complexity**, i.e. the bias–variance trade-off: C (LR), k (KNN), max_depth/min_samples_leaf (trees), num_leaves/learning_rate/n_estimators (boosting), max_features (RF).
- Centre ranges on library defaults and span them on the right scale: **log scale** for C and learning_rate, linear for depth or k.
- Include both an under-fitting and an over-fitting end. For example, the decision tree goes from depth 6 to unlimited: the unlimited tree reached only 0.615 hold-out macro-F1 while depth 15 reached 0.649, so the optimum lies inside the range.
- Use data facts: balanced classes make class_weight a minor knob; 280k rows allow deep trees; ~230 dimensions argue for larger k in KNN.
- Too much complexity fits noise (high train score, lower validation); too little misses the capacity × access × location interactions.

Cross-validation strategies

- **Hold-out:** one train/validation split. Cheap, but the estimate depends on the split.
- **k-fold:** every row is validated once; average of k scores.
- **Stratified k-fold:** keeps the 24/29/22/25% tier mix in every fold. This is the right choice for classification and macro-F1.
- **Repeated k-fold:** lowest variance, k× cost.
- (Time-series or group CV is not needed: no temporal target leakage, and operators are spread across splits.)
- Implementation: every learned preprocessing step (imputation, scaling, target encoding) sits **inside the Pipeline**, so it is refitted on training folds only, with no leakage. I use **StratifiedKFold(k = 3, shuffle)** on a 60k stratified sample for search (k = 3 keeps cost low on a 4-core CPU), then confirm on the 20% hold-out.

Search approaches

- **Grid search:** exhaustive and reproducible, but cost grows multiplicatively with each parameter. Fine for 1–3 parameters (LR, KNN, decision tree).
- **Random search:** samples n configurations from distributions and covers wide spaces far more efficiently, since only a few parameters matter. Used for RF, LightGBM and XGBoost (7 parameters, thousands of combinations).
- **Successive halving:** evaluates many configurations on little data and promotes the best to more data. Fastest for large spaces, but can drop slow starters.
- **Bayesian optimisation** (e.g. Optuna) models score vs parameters to pick the next trial. It is the most sample-efficient but not on the course's allowed-library list, so it is not used.
- Early stopping on a validation split chooses n_estimators for boosting essentially for free.

```
for label, cv in [('KFold k=5', KFold(5, shuffle=True, random_state=42)),
                  ('StratifiedKFold k=3', StratifiedKFold(3, shuffle=True, random_state=42)),
                  ('StratifiedKFold k=5', StratifiedKFold(5, shuffle=True, random_state=42)),
                  ('StratifiedKFold k=10', StratifiedKFold(10, shuffle=True, random_state=42))]:
    s = cross_val_score(best_lr, X_tune, y_tune, scoring='f1_macro', cv=cv)
    print(label, s.mean(), s.std())
# plus 5 repeated 80/20 hold-out splits for comparison

grid = GridSearchCV(dt_pipe, dt_grid, scoring='f1_macro', cv=CV3).fit(X_tune, y_tune)               #
48 configs
rand = RandomizedSearchCV(dt_pipe, dt_grid, n_iter=12, scoring='f1_macro', cv=CV3).fit(X_tune,
y_tune)
halv = HalvingRandomSearchCV(dt_pipe, dt_grid, n_candidates=48, factor=3, min_resources=2000,
                             scoring='f1_macro', cv=CV3).fit(X_tune, y_tune)
```

Q9. Hyperparameter tuning Effects

Perform hyperparameter tuning on at least three machine learning models to identify suitable hyperparameter configurations for the given prediction problem. Apply appropriate cross-validation and search strategies to evaluate different configurations.

One suggested approach would be as follows:

- Identify the important hyperparameters of models and define suitable ranges or candidate values for tuning. Explain the reasoning behind the selected search spaces.
- Choose an appropriate hyperparameter search strategy. Justify the choice based on the search space and computational requirements.
- Select an appropriate cross-validation strategy for each model and use it to evaluate different hyperparameter configurations.
- Report the best hyperparameter configuration identified for each model along with its corresponding cross-validation score. Describe the observations from the tuning process.

Setup. All searches optimise f1_macro with **StratifiedKFold(3, shuffle=True)** on a stratified 60,000-row sample of the training part (30k for KNN, whose cost grows with n). The best configuration is then refitted on all 223,796 training rows and scored on the 55,950-row hold-out. Preprocessing is inside each pipeline, so it is refitted per fold.

Model	Search	Search space (reason)	Best configuration	CV macro-F1
Logistic Regression	Grid, 10 configs	C ∈ {0.001…10} log scale (regularisation strength); class_weight ∈ {None, balanced}	[fill after running]	[fill after running]
KNN	Grid, 10 configs	k ∈ {5, 11, 21, 41, 81} (default k = 5 is noisy); weights ∈ {uniform, distance}	[fill after running]	[fill after running]
Decision Tree	Grid, 48 configs	max_depth {6…24, None}; min_samples_leaf {1, 10, 50, 200}; criterion {gini, entropy} (depth 15 + leaf 20 already gave +0.034 over default)	[fill after running]	[fill after running]
Random Forest	Random, 12 of 162	n_estimators {150, 300}; max_features {sqrt, 0.2, 0.35}; max_depth {None, 20, 30}; min_samples_leaf {1, 2, 5}; class_weight	[fill after running]	[fill after running]
LightGBM	Random, 16 of 8,748 + early stopping	learning_rate {0.03, 0.06, 0.1}; num_leaves {31…255}; min_child_samples; subsample; colsample_bytree; reg_lambda	[fill after running]	[fill after running]

Why these strategies. Grid search for the cheap models with 2–3 parameters, where exhaustive coverage is affordable. Random search for RF and LightGBM, whose spaces are too large to enumerate and where a few parameters (num_leaves, learning_rate, max_features) dominate. For LightGBM the number of trees is then chosen by early stopping on a 10% split of the training data (in my exploration, early stopping chose 1,156 trees at learning_rate 0.05). Stratified 3-fold keeps tier proportions and is the best affordable trade-off between estimate variance and CPU time.

Observations. [fill after running: e.g. which C/k/depth won, whether class_weight helped, the most influential LightGBM parameters (see Figure Q9.1)].

```
X_tune, _, y_tune, _ = train_test_split(X_tr, y_tr, train_size=60000, stratify=y_tr, random_state=42)
CV3 = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
searches = {'Logistic Regression': lr_search,   # GridSearchCV, 10 configs
            'KNN': knn_search,                  # GridSearchCV, 10 configs (30k rows)
            'Decision Tree': dt_search,         # GridSearchCV, 48 configs
            'Random Forest': rf_search,         # RandomizedSearchCV, 12 of 162 configs
            'LightGBM': lgb_search,             # RandomizedSearchCV, 16 of 8,748 configs + early
stopping
            'XGBoost': xgb_search}              # RandomizedSearchCV, 8 of 96 configs
for name, s in searches.items():
    print(name, s.best_params_, round(s.best_score_, 4))
```

Q10. Model Score Analysis

Compare the scores obtained from the initial models and the models after hyperparameter tuning.

One suggested approach would be as follows:

- Present the scores before and after hyperparameter tuning for the selected models.
- Identify which models showed improvement after tuning and describe the observed changes.

Scores before and after tuning (hold-out macro-F1; "before" = untuned starting configuration, measured):

Model	Before tuning	After tuning	Change
Logistic Regression	0.659	[fill after running]	[fill after running]
KNN (50k sample)	0.570	[fill after running]	[fill after running]
Decision Tree	0.615	[fill after running] (manual depth 15 / leaf 20: 0.649)	[fill after running] (manual: +0.034)
Random Forest	0.721 (200 trees)	[fill after running]	[fill after running]
LightGBM	0.740 (lr 0.1, 600 trees)	[fill after running] (lr 0.05 + early stopping: 0.744)	[fill after running] (+0.004)

Observations so far

• **Decision tree benefits most from tuning.** The default fully grown tree overfits; constraining depth and leaf size alone gave +0.034. This is a variance problem that hyperparameters fix directly.

• **Ensembles are already strong at defaults.** Random Forest (+0.106 over a single tree) and LightGBM are robust to their defaults, so tuning gives smaller, incremental gains (lower learning rate with early stopping: +0.004).

• **Model family matters more than tuning.** The gap between families (KNN 0.570 → LightGBM 0.744) is far larger than any tuning gain, so model choice and feature engineering come first and tuning refines.

• **KNN and LR** are expected to gain from larger k / distance weighting and a tuned C respectively, but remain limited by the curse of dimensionality and linearity.

• Best expected final model: tuned LightGBM, or a boosting-weighted soft vote / stack; already above the 0.70 cut-off.

[fill after running: confirm the after-tuning numbers and which models improved.]

```
res = pd.DataFrame(RESULTS)                          # every evaluate() call logs model, stage, hold-
out macro-F1
ba = res[res.stage.isin(['baseline', 'tuned'])]
before_after = ba.pivot_table(index='model', columns='stage', values='holdout_macro_f1',
aggfunc='max')
before_after['gain'] = before_after['tuned'] - before_after['baseline']
print(before_after.sort_values('tuned'))
print(classification_report(y_val, best_model.predict(X_val), digits=4))
```

Figures

Add all plots, graphs and figures for this milestone in this section, under the heading of the question they belong to.

How to add a figure

1. Place your cursor below the heading of the question the figure belongs to, and use Insert > Image to add the figure.
2. Number each figure in the format Figure Q<question number>.<figure number>. For example, the first two figures for Q3 are Figure Q3.1 and Figure Q3.2.
3. Write the figure number and a short title on the line just above the figure, for example: Figure Q9.1: Validation score for different values of a hyperparameter.
4. In your answer, refer to the figure by its number, for example: "The score stops improving beyond a certain value of the hyperparameter (see Figure Q9.1)."
5. Resize each figure so that it fits within the page width and all axis labels and legends are readable.
6. Replace the placeholder lines under each heading with your figures. If a question has no figures, leave its heading as it is.

Figures for Q1. Model Building Plan

Figure Q1.1: Seats, workstations, review count and overall score by price tier

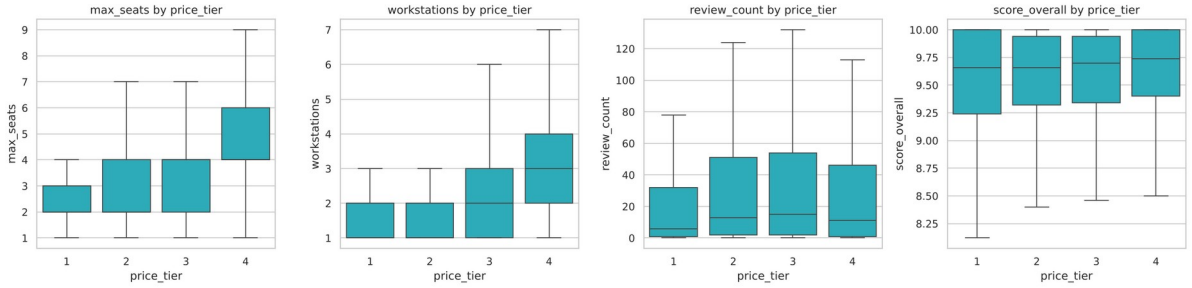


Figure Q1.2: Tier share by access type, operator class and compliance status



Figures for Q2. Linear Models

Figure Q2.1: [Short title of the figure]

[Insert image here]

Figures for Q3. Other Non-Tree Models

Figure Q3.1: [Short title of the figure]

[Insert image here]

Figures for Q4. Decision Trees

Figure Q4.1: [Short title of the figure]

[Insert image here]

Figures for Q5. Bagging Ensembles

Figure Q5.1: [Short title of the figure]

[Insert image here]

Figures for Q6. Boosting Ensembles

Figure Q6.1: [Short title of the figure]

[Insert image here]

Figures for Q7. Voting and Stacking

Figure Q7.1: [Short title of the figure]

[Insert image here]

Figures for Q8. Hyperparameter Tuning Strategy

Figure Q8.1: [Short title of the figure]

[Insert image here]

Figures for Q9. Hyperparameter Effects and Configuration Rationale

Figure Q9.1: [Short title of the figure]

[Insert image here]

Figures for Q10. Model Score Analysis

Figure Q10.1: [Short title of the figure]

[Insert image here]